

浙江大学



本科实验报告

课程名称:	C++项目管理与工程实践
实验名称:	中期总结报告
姓 名:	刘欣瓚 石耿同
专 业:	计算机科学与技术
指导教师:	袁昕
报告日期:	2026/7/14

Table of Contents

一、项目概述	4
二、项目需求分析	4
一、用户需求	4
二、核心业务规则	4
三、非功能需求	5
三、技术开发规划	5
一、总体架构	5
二、分阶段开发规划与当前进度	5
三、测试与质量规划	6
四、工具选型及核心要素	6
一、C++20	6
二、Qt 6.10.2 与 Qt Widgets	6
三、SQLite 与 QSettings	7
四、CMake、CTest 与 Qt Test	7
五、Git 与 Markdown	7
五、智能体的使用	7
一、智能体配置与使用方式	7
二、MCP Server 设置	8
三、规则文档编写	8
四、技能文档编写	8
五、每轮对话与开发记录	8
六、实际效果	9
七、现存问题	9
六、团队协作情况	10
一、分工安排	10
二、人与智能体协同工作	10
七、阶段性成果展示	11
一、任务管理主界面	11
二、任务编辑器	12
三、依赖图可视化	12
四、状态机转换	13
五、批量操作	13
六、类别管理	14
七、逾期提醒与搜索筛选	15
八、总体心得与个人感悟	15

一、 刘欣瓚	15
二、 石耿同	16

一、项目概述

SmartMate 是一款面向 Windows 的本地单用户桌面任务规划应用。项目不只解决“记录任务”的问题，还希望根据任务状态、优先级、截止时间及任务之间的 Finish-to-Start 依赖关系，回答“当前应该优先完成什么”。在产品形态上，系统规划了主窗口与桌面宠物两个 View：主窗口承担完整的任务管理与依赖分析，桌宠将以更轻量的形式展示当前任务并提供快捷操作。

本项目同时是一项严格的 MVVM 架构实践。开发过程中不仅关注功能是否可用，还要求每条业务规则都处于正确的层、每个依赖方向都能被解释、每个核心模块都能脱离图形界面进行测试。当前阶段已完成任务主流程、类别管理、任务依赖、自动规划、有向依赖图、外观设置以及从 Qt Quick/QML 到纯 Qt Widgets 的前端迁移；桌面宠物、专注计时与统计分析仍属于下一阶段工作。

二、项目需求分析

一、用户需求

传统待办工具通常只能保存任务，难以表达“完成 A 后才能开始 B”的执行约束。SmartMate 的核心用户需求可概括为：

1. 用户能够创建、查看、编辑、归档、恢复和永久删除任务。
2. 系统必须使用明确的状态机管理任务，防止出现“待办任务直接完成”等非法状态。
3. 用户可以设置优先级、截止时间、预计用时和类别，并通过搜索与组合筛选快速定位任务。
4. 用户可以为任务建立多个前置关系，系统需要识别自依赖、重复边与循环依赖。
5. 系统应根据依赖、状态、逾期、优先级和截止时间生成稳定且可解释的推荐顺序。
6. 用户应能通过有向图理解任务链、阻塞原因和跨类别上下文。
7. 任务、类别、依赖和外观偏好在程序重启后仍应保留。
8. 后续桌宠应与主窗口共享同一业务状态，而不是复制一套任务逻辑。

二、核心业务规则

任务创建时固定为待办状态，普通字段只有在待办状态下才允许编辑。状态转换采用显式命令：待办可开始或取消，进行中可完成或取消，已完成或已取消可重做或归档。任意时刻最多只能有一个进行中任务；任务开始前还必须检查其所有前置关系是否已经完成或因取消而暂时失效。

依赖关系第一版只支持 Finish-to-Start，多个前置采用 AND 语义。完成前置任务会永久满足对应关系；取消前置任务不会删除依赖边，而是使关系暂时失效并解除阻塞；若取消任务被重做，原依赖会重新生效。任务与新建依赖、批量状态变更、类别删除及永久删除等组合操作必须整批原子成功或整批失败。

“逾期”“阻塞”“可执行”“解锁数量”和“推荐位置”均是根据实时数据计算的派生状态，不写入 SQLite。这样可以避免持久化数据与真实业务状态不一致。

三、非功能需求

- 架构可维护性：严格遵守 View、Contract、ViewModel、Model Service、Repository 与 Persistence 的职责边界。
- 可测试性：领域规则不依赖 GUI；Widget 可通过 Fake Contract 测试；完整链路可使用内存 SQLite 验证。
- 数据一致性：涉及任务与依赖的复合写入必须使用事务。
- 稳定身份：跨 View 边界统一使用稳定 TaskId 和 TaskCategoryId，禁止使用列表行号或名称作为身份。
- 可部署性：正式程序仅依赖 Qt Core、Gui、Widgets、Sql 及 Windows 平台运行库，不携带 QML/Quick 运行时。
- 易用性：支持响应式表单、类型化日期与时长选择、主题切换、拖放开始任务、图形化依赖浏览及清晰的中文错误反馈。

三、技术开发规划

一、总体架构

项目采用带抽象展示契约的 MVVM。编译期依赖固定为：

Qt Widgets View → ViewModel Contracts ← ViewModel 实现 → Model Service → Repository 接口
Persistence —————→ Repository 接口

运行时数据流分为命令链和通知链：

Widget 事件 → Contract 命令 → ViewModel → Model Service → Repository 接口
Service 信号 → ViewModel 投影 → Contract 通知 → Widget 绑定

src/app 是唯一组合根，负责创建 SQLite Repository、Service、具体 ViewModel 和 Widget，并完成依赖注入。Widget 不直接接触具体 ViewModel，ViewModel 不接触 Widget 或 SQL，Persistence 只实现 Repository 接口。这一结构使测试 Fake、SQLite 实现和未来其他持久化实现可以在不修改上层代码的情况下替换。

二、分阶段开发规划与当前进度

阶段	主要目标	当前结果
基础阶段	建立领域实体、Repository、SQLite、任务 CRUD	已完成
规则阶段	状态机、依赖校验、环检测、阻塞与排序	已完成
展示阶段	任务列表、详情、编辑器、通知与外观设置	已完成
扩展阶段	类别、批量操作、原子删除、依赖图	已完成

阶段	主要目标	当前结果
前端迁移	从 Qt Quick/QML 迁移到纯 C++ Qt Widgets	已完成，QML/Quick 已清理
中期后阶段	桌面宠物、多 View 同步	待实现
后续阶段	专注计时、统计、发布回归与答辩材料	待实现

前端迁移采用六个阶段推进：先建立 Contract，再搭建 Widgets 壳与设置页，随后迁移任务主流程、类别与依赖编辑、依赖图，最后切换正式入口并删除旧 QML。该方式把一次高风险的整体替换拆成可验证的小步迁移，迁移期间始终保留功能基线。

三、测试与质量规划

当前 CTest 基线包含 22 项非 QML 测试，覆盖以下层次：

- Model：任务服务、类别服务、状态机、状态转换、取消依赖语义与排序策略。
- Persistence：SQLite Repository 和 QSettings 外观偏好。
- ViewModel：任务投影、编辑草稿、依赖编辑、依赖图、Contract 通知与外观设置。
- Widget：使用 Fake Contract 验证设置页、任务页和依赖图的绑定与命令转发。
- Integration：使用内存 SQLite 验证任务创建、任务主流程、类别依赖、依赖图和外观设置纵向链路。
- Architecture：静态检查禁止的 include、链接方向、Controller、EventBus、Service Locator、SQL 和 View API 泄漏。
- Application：正式入口的离屏启动冒烟测试。

日常验收命令为：

```
cmake --preset debug
cmake --build --preset debug
ctest --preset debug --output-on-failure
```

涉及正式入口和部署时，还需要构建 SmartMate、运行 Widgets 相关筛选测试，并执行 Release 部署脚本验证发布目录。

四、工具选型及核心要素

一、C++20

C++ 适合构建本地桌面应用，具备较好的运行效率、类型安全和资源控制能力。项目使用普通 C++ 领域类型表达 Task、状态机和依赖算法，避免为了界面绑定让领域对象继承 QObject。C++20 也为后续使用更清晰的值类型、标准容器和算法提供了基础。

二、Qt 6.10.2 与 Qt Widgets

Qt 提供跨模块一致的对象生命周期、信号槽、模型视图、数据库访问、测试和 Windows 部署工具。最终选择 Qt Widgets 作为唯一前端，主要考虑：

- 与 C++ 工程、CMake 和桌面控件生态结合直接；
- `QAbstractListModel`、`QListView`、`QStyledItemDelegate` 适合稳定 Role 驱动列表投影；
- `QGraphicsView/QGraphicsScene` 能承担依赖图绘制、缩放和平移；
- 可通过 Qt Test 在离屏环境中对控件事件和绑定进行验证；
- 发布时不需要携带 Qt QML/Quick 运行库和插件，依赖边界更容易检查。

项目明确使用 Qt 配套的 MinGW 13.1，避免与系统中其他 MinGW 版本产生 ABI 不兼容。

三、SQLite 与 QSettings

SQLite 无需独立数据库服务，适合本地单用户任务应用，并支持事务，能够保证任务与依赖边等复合操作的原子性。SQL 被严格限制在 `src/model/persistence`，Repository 接口不暴露 Qt SQL 类型。窗口外观等轻量偏好使用 `QSettings`，但其具体调用同样只位于 Persistence 层。

四、CMake、CTest 与 Qt Test

CMake 负责按层建立目标和链接边界，CTest 统一组织测试，Qt Test 提供 `QSignalSpy`、`QAbstractItemModelTester` 和控件事件模拟。工具选型的核心不只是“能编译”，而是通过 target 链接关系将架构规则变成机器可检查的约束。

五、Git 与 Markdown

Git 用于保留小步提交、迁移轨迹和问题修复记录；Markdown 用于维护功能说明、架构约束、迁移指南、Windows 部署说明和本报告。代码、规则、测试与文档能够在同一仓库中共同评审，有利于追溯每次设计决定。

五、智能体的使用

一、智能体配置与使用方式

开发过程采用“开发者给出目标与验收要求，智能体读取仓库上下文后执行小步修改”的协作模式。智能体使用 Codex，使用 PowerShell 操作 CMake、Git 和测试工具。每轮任务通常遵循以下流程：

1. 读取 `AGENTS.md`、相关架构文档和目标模块代码。
2. 在修改前声明所属层、目标数据流和测试方式。
3. 检查 Git 工作区，避开用户未提交的无关修改。
4. 以最小改动实现目标，并同步修改邻近注释和文档。
5. 先运行相关测试定位问题，再执行完整配置、构建和 CTest。
6. 汇报变更文件、验证结果、已知限制和下一步建议。

智能体并不被允许自由改变架构。它必须遵守固定依赖方向、稳定 ID、原子 Repository 端口以及 Widget Fake 测试等规则。这种配置将智能体定位为“受工程约束的协作开发者”，而不是一次性代码生成器。

二、MCP Server 设置

仓库中没有 MCP Server 配置，也没有依赖某个远程 MCP 服务才能完成构建。当前开发的核心能力来自本地文件访问、PowerShell、CMake/CTest 和 Git，因此其他开发者克隆仓库后仍可独立复现项目。

三、规则文档编写

AGENTS.md 是本项目最重要的智能体规则文档。它将架构原则转化为可执行约束，主要包括：

- 固定编译期依赖与运行时数据流；
- Qt Widgets 为唯一正式前端，禁止重新引入 QML、Quick 和 .ui；
- Common、Model、Persistence、Contracts、ViewModel、Widget 和 App 的职责；
- 稳定 ID、强类型命令、通知信号、初始同步和双向绑定规则；
- 任务状态机、依赖、类别、批量事务和图布局的归属；
- 中文注释规范、修改前说明和修改后验收命令；
- 禁止 Controller、EventBus、Service Locator、全局业务单例和跨层捷径。

与简短提示词相比，规则文档的价值在于它长期存在于仓库中，可被每轮智能体会话重新读取，也能被人工评审和架构测试交叉验证。

四、技能文档编写

现阶段 的技能文档主要分散在 AGENTS.md、docs/architecture.md、docs/widgets-migration.md 和 docs/windows-deployment.md 中。这些文档已经能约束开发，指导开发。

五、每轮对话与开发记录

日期	轮次依据	对话目标/任务	形成的结果
07-10	ddd6458	初始化严格 MVVM 项目	建立分层目录、构建目标和基础边界
07-10	78f373a	补充项目说明	增加中文指南和 Windows 部署文档
07-11	1820f6c	建立任务列表投影	实现 TaskListViewModel 与任务管理绑定
07-11	e15050c	增加查询与排序	实现搜索、排序策略及测试
07-11	cb26d91	建立任务依赖	实现依赖管理基础能力
07-11	8103ba3	扩展创建与图投影	增加前置选择和 TaskGraphViewModel
07-11	3b45817	修复归档编辑漏洞	阻止未恢复的归档任务被编辑
07-11	6212176	固化状态转换	实现状态机、转换服务与综合测试
07-12	bb76760	增加外观设置	实现设置持久化与界面设计更新

日期	轮次依据	对话目标/任务	形成的结果
07-12	9c09ab9	丰富依赖图数据	增加前置/后继模型与投影
07-12	57abe4b	整理代码边界	优化结构并保持 MVVM 约束
07-12	8308142	修复图布局	提升依赖图布局与响应式表现
07-12	3fc2f1d	增加删除和逾期	实现永久删除、依赖清理与逾期策略
07-13	81264ad	增加批量转换	实现整批原子状态操作
07-13	3a52e8e	增加任务类别	完成类别领域、服务和持久化能力
07-13	cbaac08	启动 Widgets 迁移	从 Qt Quick/QML 转向 Qt Widgets
07-13	c99d6cc	隔离 View 与实现	引入 ViewModel Contracts 和通知体系
07-13	38c10b1	建立 Widgets 主框架	实现主窗口结构与组合方式
07-13	2e36938	迁移任务主流程	完成任务页面、详情、编辑和命令链
07-13	90202fd	迁移类别与依赖编辑	完成类别管理、筛选和原子依赖保存
07-14	72444d1	迁移依赖图	实现 DependencyGraphView 和图元
07-14	8954a37	修复迁移后布局	处理页面布局回归
07-14	b35a2e5	优化任务界面	增强响应式编辑器、任务页和主题
07-14	926b7c7	改善类型化输入	完善截止日期和预计时长选择器
07-14	05cfdc4	完成迁移清理	删除 QML 测试、旧代码和无用依赖

六、实际效果

智能体的主要正面效果体现在以下方面：

- 能够在较短周期内完成从领域规则、持久化、ViewModel 到 Widget 的纵向功能闭环。
- 能根据编译错误和测试失败持续迭代，而不是只生成静态代码片段。
- 在大规模前端迁移中维持阶段性目标和回归基线，最终删除旧 QML 依赖。
- 能同步维护架构、迁移和部署文档，使代码决策具备文字依据。
- 通过 Fake、信号测试、内存 SQLite 和架构守卫提高回归发现能力。

七、现存问题

1. 智能体生成的界面需要人工进行视觉验收，自动化测试难以覆盖间距、色彩和高 DPI 下的全部问题。
2. 长任务中可能出现上下文偏移，必须依靠窄任务、阶段性提交和完整测试限制风险。
3. 当前桌宠、专注计时和统计尚未实现，产品闭环仍不完整。

六、团队协作情况

一、分工安排

本项目由两名成员协作完成。在开发过程中，团队采用了“人机协同”的工作模式——人类成员负责架构决策与领域设计，智能体辅助完成编码实现、测试编写。本项目中，刘欣瓚同学主要负责核心代码的产出，石耿同同学主要负责功能性的测试验证

二、人与智能体协同工作

在整个开发周期中，团队成员与智能体形成了紧密的协作关系。具体实践如下：

Info

架构决策由人主导。MVVM 分层边界、状态机转换规则、依赖图的 Finish-to-Start 语义、批量操作的原子事务策略等核心设计均来自团队成员的讨论与判断。智能体不参与架构决策，而是提供“这个设计会导致什么问题”的逆向检验。

Tip

编码实现由智能体高效执行。在架构确定后，具体的 C++ 实现由智能体完成——包括 Service 方法、ViewModel 属性绑定、Qt Widgets 组件布局。人类成员的角色转变为 Code Review：审查生成的代码是否遵守 AGENTS.md 中定义的 MVVM 边界，是否正确处理边界条件。

Warning

测试用例由人和智能体共同设计。人类提出测试场景（如“取消任务不阻塞后继”、“批量归档含不合格任务整批回滚”），智能体编写具体的测试代码（Fake Repository 注入、断言编写）。手工功能测试表由人设计操作步骤，智能体补充边界场景。

协同效果：

1. 效率提升：大量重复性编码工作（如 WidgetBinding 双向同步、SQL 参数绑定、QAbstractListModel 角色映射）由智能体完成，人类专注于设计决策和质量把关。
2. 质量保障：智能体能够精准执行 MVVM 分层规则，在代码生成时自动检查依赖方向，避免了“随手把 SQL 写到 ViewModel”这类常见错误。
3. 知识传递：智能体对 Qt/C++/CMake 的最佳实践有全面掌握，能够在生成代码时推荐合适的 API 用法（如 `beginResetModel()/endResetModel()` 协议、`QDateTime::TransitionResolution::Reject` 等），团队成员在 Review 过程中同步学习。

现存的协同挑战：

1. 智能体生成测试文件无法覆盖到所有边界条件，仍需人类成员手工补充。
2. 依赖图可视化（正交折线路由、节点布局）这类算法问题，智能体的第一次输出往往不理想，需要多轮迭代。

3. 智能体对架构的理解往往不够深入,容易在实现中出现“看似合理但违反设计原则”的代码,需要人类成员在 Review 中指出。

七、阶段性成果展示

一、任务管理主界面

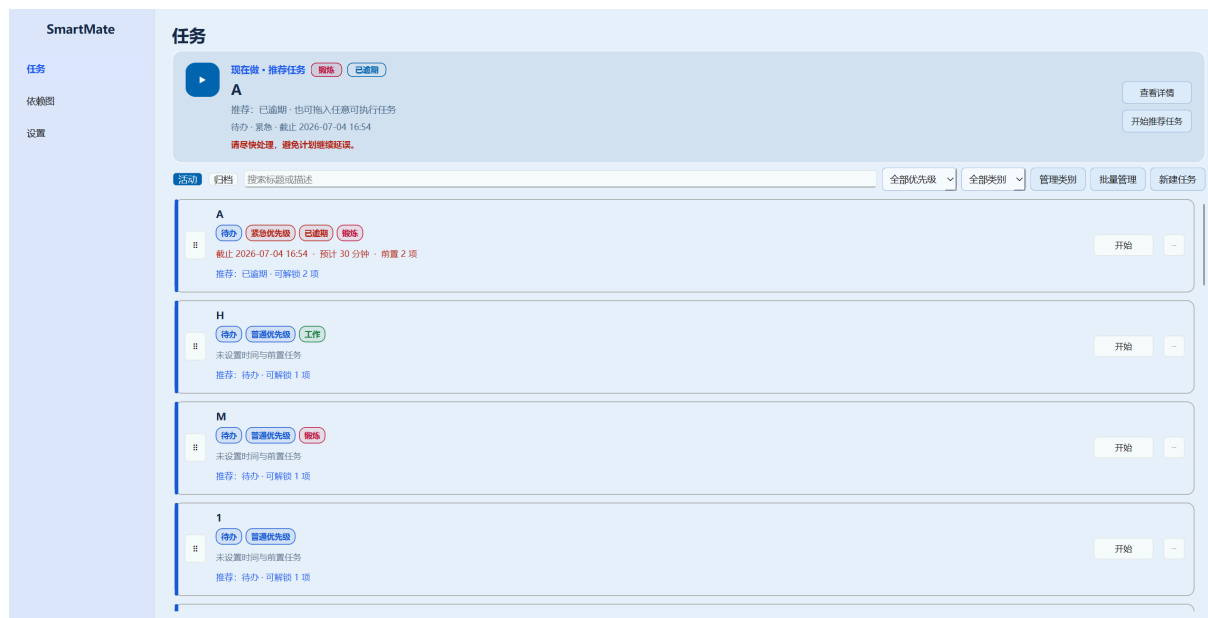


Figure 1: 任务管理主界面 (Qt Widgets 版本)。顶部为“现在做”聚焦槽,展示系统推荐的最优先任务并支持拖拽开始;中部为搜索与筛选工具栏,包含关键字搜索、优先级筛选、类别筛选和批量管理入口;下方为 QListView 任务卡片列表,卡片由 TaskItemDelegate 自绘,左侧色条表示任务状态,徽标式标签展示状态、优先级、逾期和类别信息。

二、任务编辑器



Figure 2: 任务编辑器。状态为只读展示，新建任务固定为“待办”。可设置优先级、类别、前置任务、截止时间与预计用时。类别通过下拉框选择，支持跳转到类别管理弹窗。

三、依赖图可视化

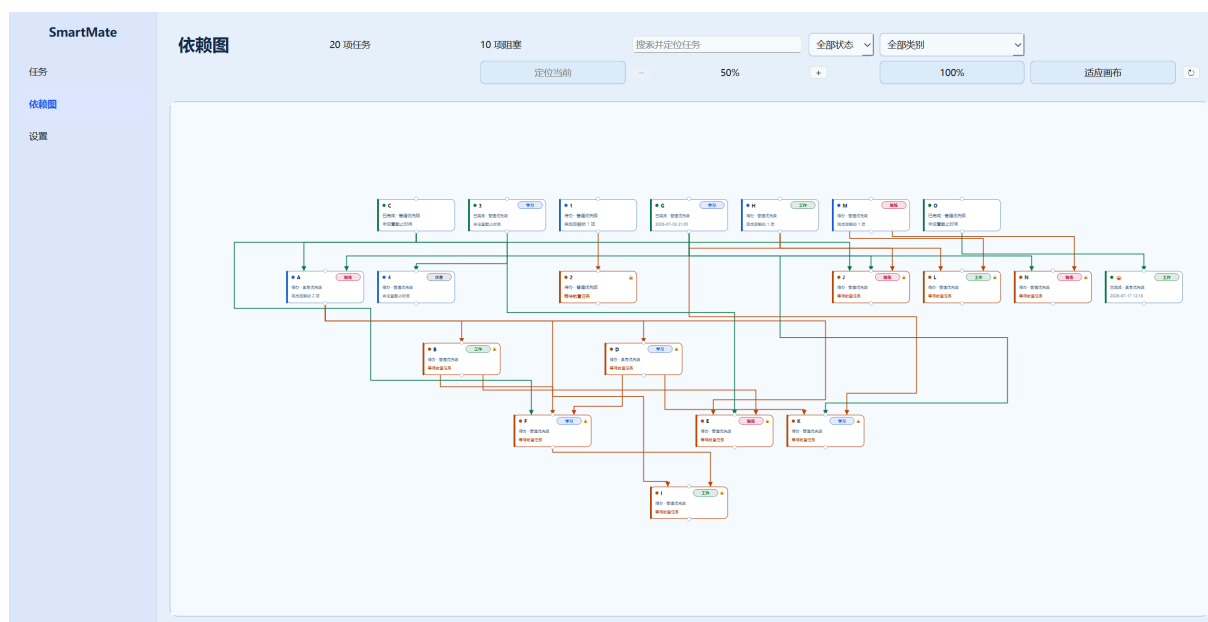


Figure 3: Finish-to-Start 依赖图页面，使用 QGraphicsView + QGraphicsObject 渲染。节点按 Kahn 拓扑层级从左到右排列，边使用正交折线路由，绿色箭头表示前置已满足，橙色箭头表示未满足。右侧面板展示选中节点的直接前置、后继与阻塞原因。支持类别筛选——选中类别后核心节点高亮，跨类别邻居节点半透明显示。

四、状态机转换

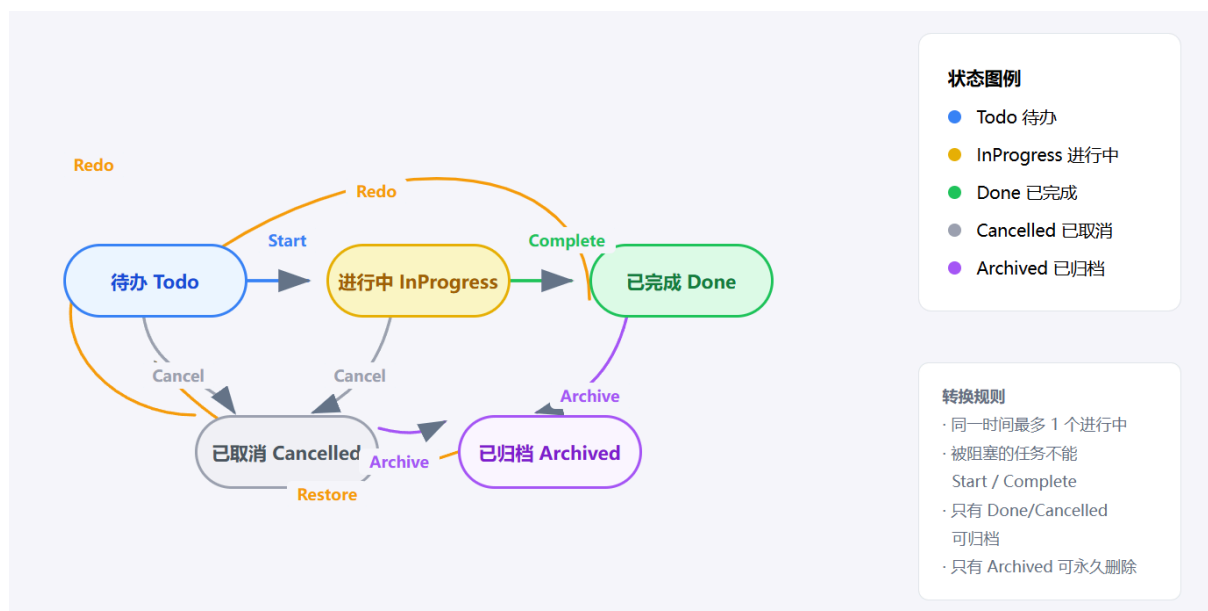


Figure 4: 任务状态机转换图。六个转换命令：Start (Todo→InProgress)、Complete (InProgress→Done)、Cancel (Todo/InProgress→Cancelled)、Redo (Done/Cancelled→Todo)、Archive (Done/Cancelled→Archived)、Restore (Archived→Todo/原状态)。核心约束：同一时间最多一个进行中；被阻塞任务不能 Start/Complete；只有 Done/Cancelled 可归档；只有 Archived 可永久删除。

五、批量操作

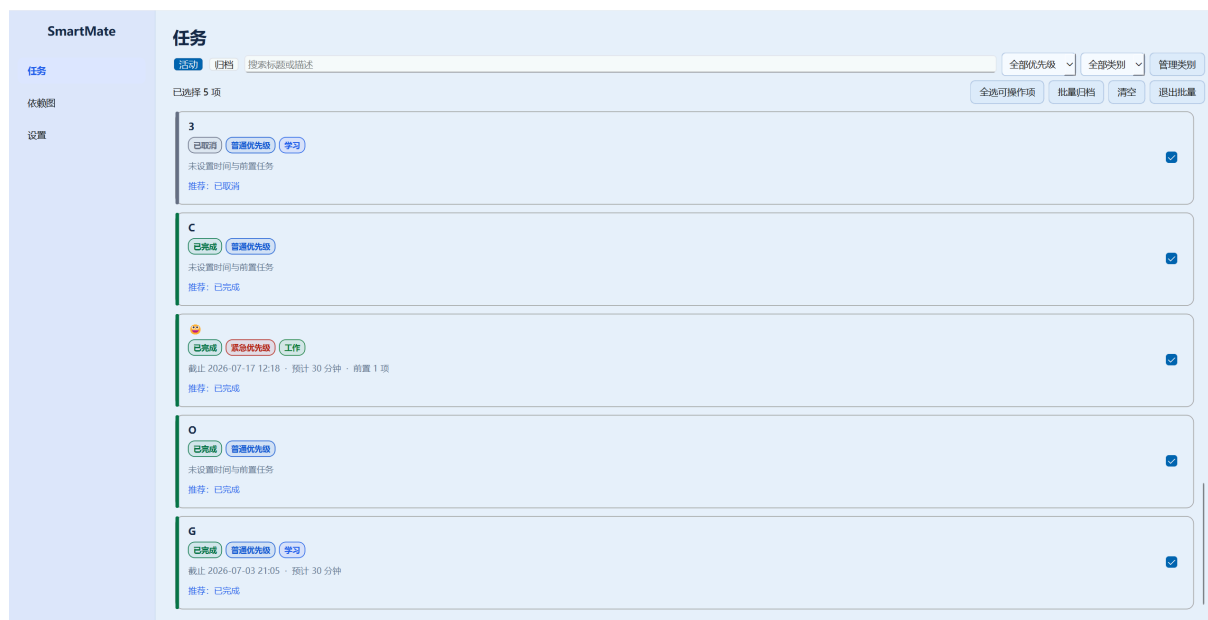


Figure 5: 批量操作模式 (Qt Widgets 版本)。进入批量模式后，聚焦槽和卡片操作按钮隐藏，卡片右侧出现复选框，顶部工具栏切换为批量操作栏。支持全选/清空、批量归档（活动视图）、批量恢复和批量永久删除（归档视图）。批量命令通过数据库条件 UPDATE 和事务回滚保证原子性——任一任务不合格则整批不写入。

六、类别管理

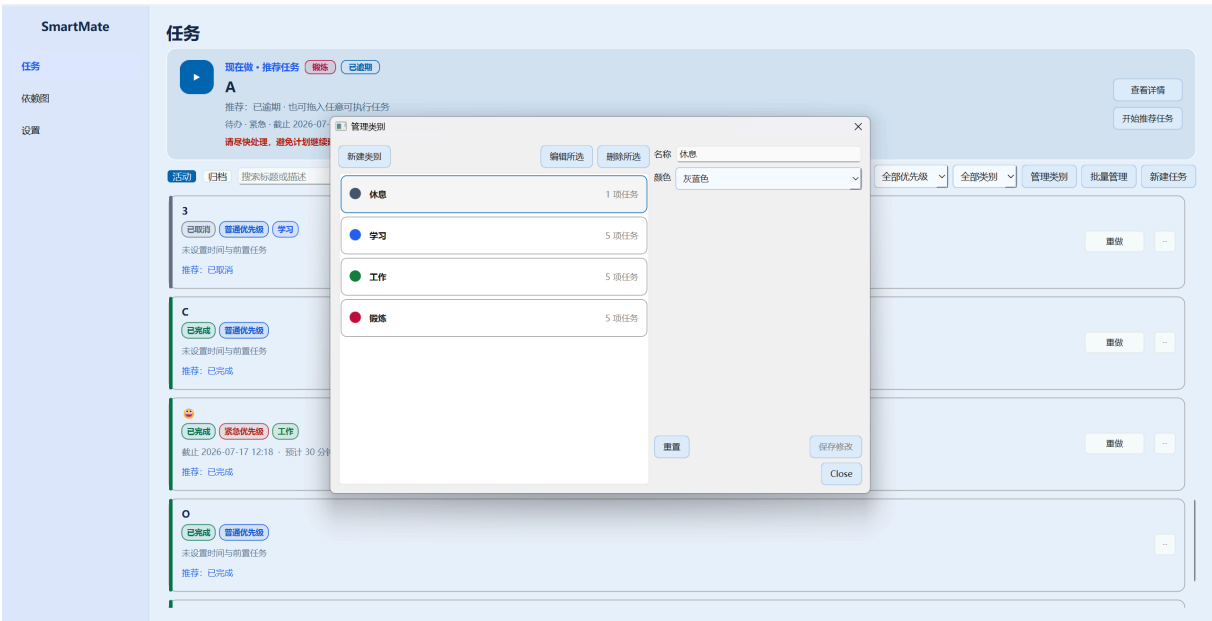


Figure 6: 类别管理弹窗。左侧为已有类别列表，显示颜色标识、名称与关联任务数。右侧为创建/编辑表单，包含名称输入框和 8 色调色板。支持创建、编辑和删除类别，删除有任务关联的类别时自动将任务归为“未分类”。

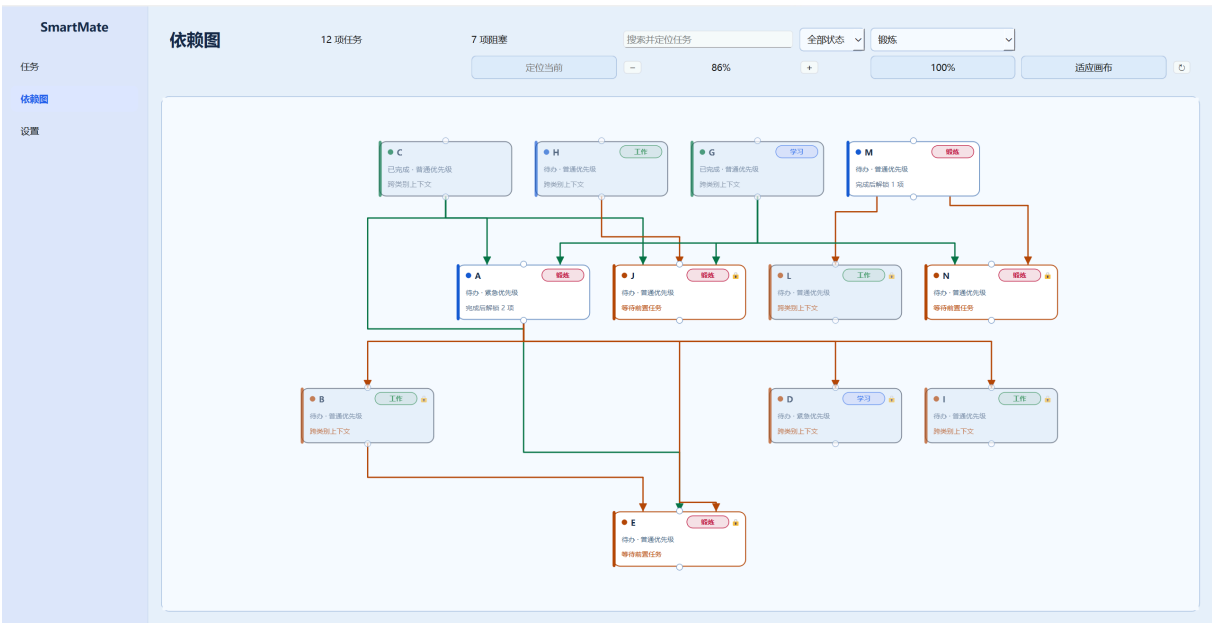


Figure 7: 依赖图中也可以用类别来显示。其中白色的为该类别的任务，浅色的为该类别任务绑定的相关依赖任务。“。

七、逾期提醒与搜索筛选

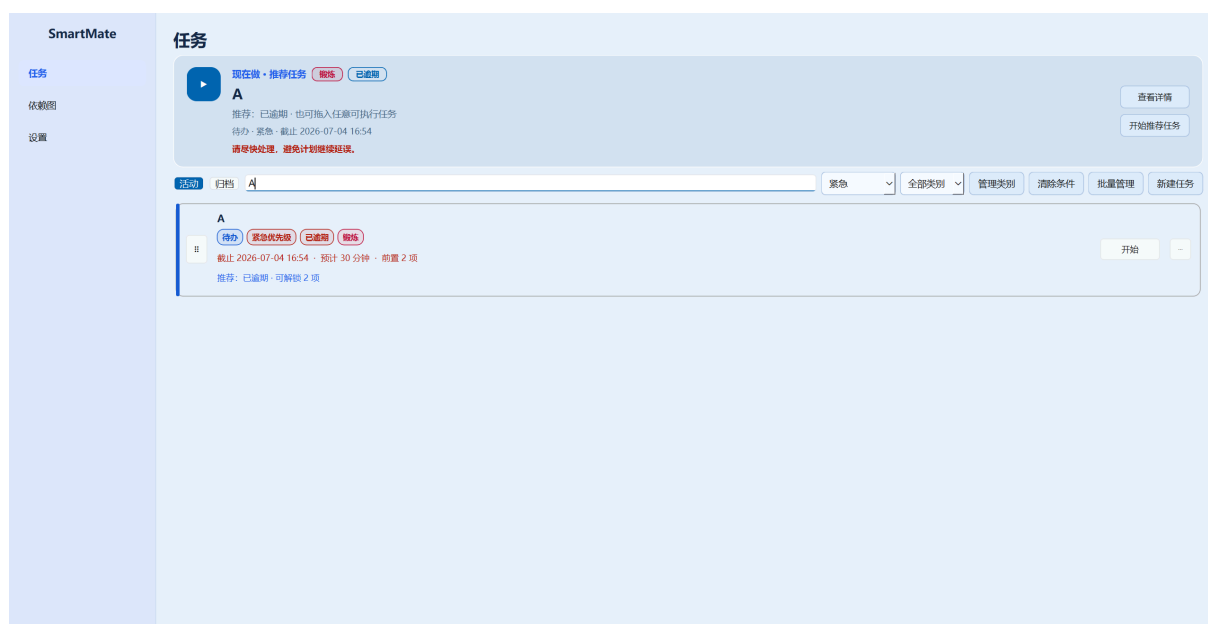


Figure 8: 逾期提醒与搜索筛选。逾期的 Todo/InProgress 任务卡片在标题行显示红色“已逾期”圆角徽标，元数据行中的截止时间文字同步变红。搜索框支持关键字实时匹配标题和描述，优先级下拉和类别下拉支持独立或组合筛选，活动/归档标签切换视图。筛选条件改变时列表即时刷新，清除条件一键恢复。

八、总体心得与个人感悟

一、刘欣瓚

项目推进过程中最明显的体会是，功能数量并不是衡量进度的唯一标准，能够稳定修改、能够解释错误同样重要。前期若只追求界面快速可见，很容易把状态规则、数据库写入和按钮逻辑混在一起；当依赖、批量操作和第二个 View 出现后，这种耦合会迅速放大。解耦不是让模块之间完全没有联系，而是让联系通过稳定接口发生，并使依赖指向更稳定的一方。本项目中的 Repository 接口隔离了 Service 与 SQLite, Contract 隔离了 Widget 与具体 ViewModel，组合根集中处理具体对象的创建。同时我也认识到，解耦并不等于接口越多越好。接口必须围绕真实变化方向设计。

通过这个项目，我对 MVVM 的理解从把界面和数据分成几个目录转变为建立可验证的数据流和依赖方向。ViewModel 不是万能的中间层，它不能接管业务规则，也不能操纵 Widget；它真正的职责是把 Model 状态转换为 View 容易观察的投影，并把用户意图转换为明确命令。Model 不知道界面如何呈现，View 不知道规则如何计算，Contract 则为二者提供稳定、窄小、可测试的边界。

本阶段最大的成长是开始用“规则放在哪里、谁拥有状态、失败如何回滚、变化如何通知、测试如何证明”来审视代码，而不再只关注界面是否运行，这使我对项目开发有了全新的认识与感受。需要反思的是，项目早期使用 QML 实现 view，容易造成耦合，所以

不得不重构使用 Widgets 实现，耗费了更多的时间精力。对于大项目，在开始前就应当充分思考，避免后期再推翻重来。智能体的使用提高了实现速度，但不能替代设计责任和人工验收。高质量协作的关键不是给出一句模糊需求后直接接受结果，而是提供明确规则、要求智能体说明数据流、让测试验证行为，并由开发者最终判断架构取舍和用户体验。今后还应加强技能沉淀和阶段复盘，使智能体协作从个人开发便利提升为可复现的工程流程。

二、石耿同

这个项目我主要负责测试和验证工作。通过与智能体的协作，我对“测试驱动开发”有了更深刻的理解。最初我只是把测试当作“验证功能是否正确”的手段，但在项目中，我逐渐意识到测试本身也是一种设计工具——通过编写测试用例，我可以提前思考各种边界条件和异常场景，从而推动架构设计的完善。

另一个重要收获是关于 MVVM 架构的实践经验。前期理论课中我对 MVVM 只有大致印象：Model 提供数据和业务规则，ViewModel 管理命令并将数据与 View 绑定，View 负责渲染。但在实际开发中，我们发现第一版使用 QML 作为 View 层时，QML 的属性绑定机制虽然便捷，但隐式耦合了 ViewModel 的具体实现。因此中期决定将 View 层重构为 Qt Widgets，并引入 Contract 抽象层——View 只依赖纯虚接口，不直接感知具体 ViewModel 实现。这次重构让我深刻体会到：MVVM 的核心不是某一套 UI 框架，而是依赖方向——**View** 不应该知道自己绑定的是哪个 **ViewModel**。通过严格遵循这一原则，不仅提高了代码的可维护性，也让同一套 ViewModel 可以同时驱动 QML 和 Widgets 两套 View，为后续的功能扩展（如桌宠独立窗口）打下了基础。